

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2-PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Raiyan I	Reg.No.:	192521421
Department:	Btech IT	Date:	31/8/2026

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I G Devendra Nath Reddy (192425204) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding(20%)	Algorithm Design(30%)	Use of Control Structures(20%)	Pseudocode Structure(10%)	Completeness(20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS

1. DMA-Based Data Transfer Algorithm

Objective

Design an algorithm that transfers data from an I/O device directly to memory using **DMA (Direct Memory Access)** while using interrupts to notify the CPU after completion.

Pseudocode

ALGORITHM DMA_Data_Transfer

INPUT:

Device_Address
Memory_Address
Transfer_Size

OUTPUT:

Transfer_Status

BEGIN

1. Initialize DMA Controller
2. Set DMA Source Address
DMA.Source $\leftarrow$ Device_Address
3. Set DMA Destination Address
DMA.Destination $\leftarrow$ Memory_Address
4. Set Transfer Size
DMA.Length $\leftarrow$ Transfer_Size
5. Configure DMA Mode
DMA.Mode $\leftarrow$ DEVICE_TO_MEMORY
DMA.Interrupt $\leftarrow$ ENABLED
6. Clear Previous DMA Status
DMA.Status $\leftarrow$ CLEAR
7. Start DMA Transfer
DMA.Control $\leftarrow$ START
8. CPU performs other tasks
WHILE DMA.Status = BUSY DO

```
CPU.Execute_Other_Tasks()
END WHILE
```

9. IF DMA generates INTERRUPT THEN

```
    Save CPU Context
```

```
    Check DMA Status
```

```
    IF DMA.Status = TRANSFER_COMPLETE THEN
```

```
        Transfer_Status ← SUCCESS
```

```
    ELSE IF DMA.Status = ERROR THEN
```

```
        Transfer_Status ← FAILURE
```

```
        Handle_DMA_Error()
```

```
    END IF
```

```
    Clear DMA Interrupt
```

```
    Restore CPU Context
```

```
END IF
```

10. Return Transfer_Status

```
END
```

Explanation

DMA allows the **I/O device to transfer data directly into main memory** without requiring the CPU to copy every byte.

The normal transfer is:

I/O Device

|
| Data
↓

DMA

|
↓

Main Memory

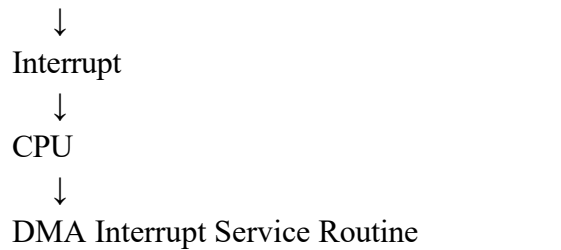
The CPU only performs control operations such as configuring the source address, destination address, and transfer size.

When the transfer finishes, the DMA controller generates an interrupt:

DMA Transfer

↓

Transfer Complete



Why DMA minimizes CPU involvement

Without DMA:

Device → CPU → Memory

The CPU handles every transfer.

With DMA:

Device → DMA → Memory

↓
CPU
only handles
completion

Therefore, DMA is especially useful for **high-speed data acquisition, disk transfers, network packets, audio, and video.**

Advantages

- Reduces CPU workload
- Supports high-speed data transfer
- CPU can execute other tasks during transfer
- Interrupt indicates completion
- Suitable for large blocks of data

2. Vectored Interrupt Handling for Multiple Devices

Objective

Design an interrupt handling system that receives interrupts from multiple devices, determines their priority, and dispatches the appropriate ISR using **vectored interrupts**.

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

INPUT:

Interrupt_Request

OUTPUT:

Interrupt_Service_Status

BEGIN

1. Initialize Interrupt Controller

2. Create Interrupt Vector Table

```
Vector_Table[IRQ1] ← ISR_Device1  
Vector_Table[IRQ2] ← ISR_Device2  
Vector_Table[IRQ3] ← ISR_Device3  
Vector_Table[IRQ4] ← ISR_Device4
```

3. Assign Priorities

```
Priority[IRQ1] ← HIGH  
Priority[IRQ2] ← HIGH  
Priority[IRQ3] ← MEDIUM  
Priority[IRQ4] ← LOW
```

4. Enable Required Interrupts

5. WAIT for interrupt request

6. IF Interrupt_Request occurs THEN

Save CPU Context

Determine pending interrupt

Select highest-priority IRQ

Vector ← Vector_Table[Selected_IRQ]

Jump to Vector

Execute corresponding ISR

Clear interrupt flag

Restore CPU Context

Return from interrupt

END IF

END

Interrupt Service Routine

ISR_Device1:

BEGIN

Save Required Registers

Read Device Status

Process Critical Data

Clear Device Interrupt

Set Background_Task_Flag if required

Restore Registers

RETURN

END

Explanation

A **vectored interrupt** provides the CPU with a specific vector associated with the interrupting device.

For example:

Device 1 → IRQ1 → Vector 1 → ISR_Device1

Device 2 → IRQ2 → Vector 2 → ISR_Device2

Device 3 → IRQ3 → Vector 3 → ISR_Device3

Therefore, the CPU does not need to check every device individually.

Interrupt Vector Table

Interrupt	Device	Priority	ISR
IRQ1	Sensor	High	ISR_Device1
IRQ2	Network	High	ISR_Device2
IRQ3	USB	Medium	ISR_Device3
IRQ4	Background Device	Low	ISR_Device4

Priority Handling

If multiple interrupts arrive simultaneously:

IRQ1 → HIGH

IRQ2 → HIGH

IRQ3 → MEDIUM

IRQ4 → LOW

The interrupt controller selects the highest-priority pending interrupt.

For example:

IRQ3 executing

↓

IRQ1 arrives

↓
 Save IRQ3 context
 ↓
 Execute ISR1
 ↓
 Restore IRQ3 context
 ↓
 Continue IRQ3

This provides **nested interrupt support**.

Advantages

- Fast interrupt dispatch
- Appropriate ISR selected directly
- Supports device prioritization
- Suitable for real-time systems
- Reduces interrupt identification overhead

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Objective

Select the appropriate I/O addressing method based on **latency and bandwidth requirements**.

Basic Concepts

Memory-Mapped I/O (MMIO):

Device registers are assigned addresses in the normal memory address space.

CPU → Memory Address → Device Register

I/O-Mapped I/O:

A separate I/O address space is used for device communication.

CPU → I/O Address → Device Register

Pseudocode

ALGORITHM Select_IO_Method

INPUT:

Required_Latency
 Required_Bandwidth
 Device_Type
 CPU_Architecture

OUTPUT:

Selected_IO_Method

BEGIN

1. Define latency and bandwidth thresholds

LOW_LATENCY_THRESHOLD $\leftarrow$ 10 microseconds

HIGH_BANDWIDTH_THRESHOLD $\leftarrow$ 100 MB/s

2. IF Required_Latency $\leq$ LOW_LATENCY_THRESHOLD THEN

 Selected_IO_Method $\leftarrow$ MEMORY_MAPPED_IO

3. ELSE IF Required_Bandwidth $\geq$ HIGH_BANDWIDTH_THRESHOLD THEN

 Selected_IO_Method $\leftarrow$ MEMORY_MAPPED_IO

4. ELSE IF Device_Type = LEGACY_IO_DEVICE THEN

 Selected_IO_Method $\leftarrow$ IO_MAPPED_IO

5. ELSE IF CPU_Architecture supports efficient MMIO THEN

 Selected_IO_Method $\leftarrow$ MEMORY_MAPPED_IO

6. ELSE

 Selected_IO_Method $\leftarrow$ IO_MAPPED_IO

7. Return Selected_IO_Method

END

Improved Decision Algorithm

ALGORITHM Dynamic_IO_Selection

BEGIN

 Measure_Device_Latency()

 Measure_Required_Bandwidth()

 IF Latency_Requirement = VERY_HIGH THEN

 Method $\leftarrow$ MEMORY_MAPPED_IO

 ELSE IF Bandwidth_Requirement = VERY_HIGH THEN

 Method $\leftarrow$ MEMORY_MAPPED_IO

 ELSE IF Device = LEGACY_DEVICE THEN

 Method $\leftarrow$ IO_MAPPED_IO

```

ELSE
    Method ← MEMORY_MAPPED_IO
END IF

```

```

Configure_IO_Controller(Method)

```

```

RETURN Method

```

END

Explanation

Memory-mapped I/O is generally preferred when the system needs:

- Low latency
- High-frequency register access
- High-performance device communication
- Integration with normal load/store instructions

I/O-mapped I/O can be useful when:

- A processor provides a separate I/O address space
- Legacy peripherals need isolated I/O addressing
- Compatibility with existing hardware is important

Example

Suppose:

$$Latency = 5\mu s$$

and:

$$Bandwidth = 500MB/s$$

Since both latency and bandwidth requirements are high:

Required Latency → HIGH

Required Bandwidth → HIGH



MEMORY-MAPPED I/O

Decision Table

Requirement	Preferred Method
Very low latency	MMIO
High bandwidth	MMIO
Frequent register access	MMIO
Legacy device	I/O-mapped
Separate I/O address space required	I/O-mapped
Modern high-performance device	MMIO

Important Note

The actual latency depends on the **processor architecture, bus, cacheability/ordering rules, device, and implementation**. Therefore, a real system should benchmark or use platform-specific specifications rather than relying only on fixed thresholds.

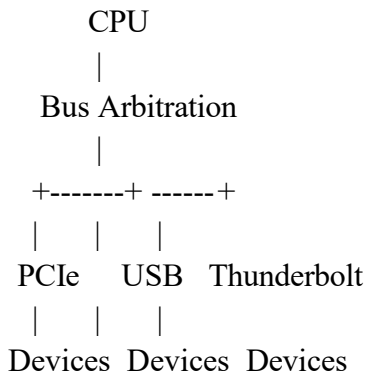
4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt

Objective

Manage multiple devices requesting bus access while ensuring:

- Fairness
- Priority
- No starvation
- Efficient bandwidth utilization

Architecture



Pseudocode

ALGORITHM Bus_Arbitration

INPUT:

Device_Request_List
Device_Priority
Device_Interface

OUTPUT:

Granted_Device

BEGIN

1. Initialize Round_Robin_Pointer

2. WHILE System_Is_Running DO

 Check all pending device requests

 IF no request exists THEN

 Continue monitoring

 END IF

 FOR each device in Device_Request_List DO

IF device has pending request THEN

 Calculate Effective_Priority

 Effective_Priority $\leftarrow$
 Base_Priority + Aging_Value

END IF

END FOR

Select device with highest Effective_Priority

IF multiple devices have same priority THEN

 Apply Round_Robin_Selection()

END IF

Grant bus access

Allow device to transfer data

Limit transfer using Time_Quantum

Revoke bus access after Time_Quantum

Update Round_Robin_Pointer

Increase aging value of waiting devices

Reset aging value of selected device

END WHILE

END

Explanation

Suppose the system has:

PCIe Device 1

PCIe Device 2

USB Device 1

USB Device 2

...

USB Device 6

Thunderbolt Device 1

Thunderbolt Device 2

Every device may request communication resources.

The arbiter determines which request is serviced first.

Priority and Fairness

A simple fixed-priority system can cause starvation:

High Priority Device

↓

High Priority Device

↓

High Priority Device

↓

Low Priority Device waits forever

Therefore, **priority aging** can be used.

If a low-priority device waits for a long time:

$$\text{Effective Priority} = \text{Base Priority} + \text{Aging}$$

Its effective priority increases until it receives service.

Round-Robin Arbitration

For devices with similar priority:

PCIe → USB → Thunderbolt → PCIe → USB → Thunderbolt

Each gets a chance to transmit.

Example:

Device A → Transfer

Device B → Transfer

Device C → Transfer

Device A → Transfer

Device B → Transfer

Device C → Transfer

This prevents one device from monopolizing the communication resource.

Time Quantum

Each device can be assigned a maximum service period:

Device A → 100 μs

Device B → 100 μs

Device C → 100 μs

After the time quantum expires, the arbiter selects another waiting device.

Advantages

- Prevents starvation
- Provides fair access
- Supports different device priorities

- Controls bus occupancy
- Useful for mixed-speed communication systems

5. Hybrid I/O Communication Mechanism

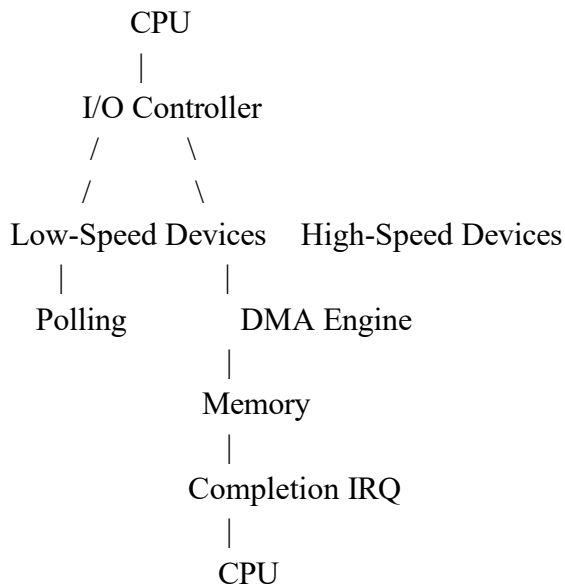
Objective

Use:

- **Polling** for low-speed devices
- **DMA + interrupts** for high-speed devices

This avoids wasting CPU resources while maintaining responsiveness.

Architecture



Pseudocode

ALGORITHM Hybrid_IO_System

INPUT:

Device_List

OUTPUT:

Device_Communication_Status

BEGIN

1. Initialize all devices

2. FOR each Device in Device_List DO

 IF Device.Speed < SPEED_THRESHOLD THEN

 Device.Mode ← POLLING

ELSE

Device.Mode ← DMA_WITH_INTERRUPT

END IF

END FOR

3. Start I/O Processing

4. WHILE System_Is_Running DO

FOR each Device in Device_List DO

IF Device.Mode = POLLING THEN

Poll_Device(Device)

IF Device.Data_Available = TRUE THEN

Read_Device_Data(Device)

Store_Data(Device)

END IF

ELSE IF Device.Mode = DMA_WITH_INTERRUPT THEN

IF DMA_Not_Configured(Device) THEN

Configure_DMA(Device)

Set_Source_Address(Device)

Set_Destination_Address(Device)

Set_Transfer_Size(Device)

Enable_DMA_Interrupt(Device)

Start_DMA(Device)

END IF

END IF

END FOR

Check_DMA_Interrupts()

END WHILE

END

DMA Interrupt Handler

ISR_DMA_Completion(Device)

BEGIN

Save_Context()

Check_DMA_Status(Device)

IF Transfer_Completed = TRUE THEN

Mark_Buffer_Ready(Device)

Clear_DMA_Interrupt(Device)

Schedule_Data_Processing(Device)

ELSE IF Transfer_Error = TRUE THEN

Handle_DMA_Error(Device)

END IF

Restore_Context()

RETURN

END

Low-Speed Device Processing

For a low-speed device, polling is acceptable.

Example:

CPU

↓

Check Sensor

↓
Data available?
↓
NO → Continue
↓
YES
↓

Read Data

For example, a slow temperature sensor may not need an interrupt for every small update.

Pseudocode

PROCEDURE Poll_Device(Device)

BEGIN

 Status ← Read_Status_Register(Device)

 IF Status = DATA_READY THEN

 Data ← Read_Data_Register(Device)

 Store_Data(Data)

 END IF

END

High-Speed Device Processing

High-speed devices should use DMA.

High-Speed Device

↓
DMA
↓
Main Memory
↓
Transfer Complete
↓
Interrupt
↓
CPU

The CPU does not need to copy every byte.

Example

If a device produces:

500MB/s

and a DMA buffer is:

$$1MB$$

then the approximate number of buffer completions per second is:

$$\frac{500MB/s}{1MB} = 500$$

So the CPU may only need to handle roughly **500 DMA completion events per second**, rather than hundreds of thousands of individual I/O operations.